청약 플랫폼 DB 가이드

두 브랜치의 DB를 병합한 결과와, 서비스용으로 전처리한 가공 레이어를 설명하는 문서입니다.

기준일	2026-09-30
브랜치	`jiyoon(cheongyakP)` — 커밋 `a4d3033` (병합) · `9a3b06e` (수정) · `8f478f1` (가공 레이어). main에는 아직 반영하지 않았습니다.
DB	PostgreSQL `cheongyak` (localhost:5432), 약 166MB
읽는 순서	처음이면 1 → 2 → 5 → 7 순서로 읽고, 컬럼을 찾을 때는 3·4를 사전처럼 보세요.

목차

1 **한눈에 보기**
무엇을 했고, 데이터가 얼마나 있는지 한 페이지 요약

2 **전체 구조** 원본 가공
API → 원본 테이블 → 가공 레이어 → 서비스 흐름, 코드 파일별 역할, 자동 갱신 주기

3 **원본 테이블 사전** 원본
listings · historical_results · loan_products · loan_rate_options의 컬럼 의미와 채움률

4 **이전 DB와 비교** 병합
두 사람의 기존 DB 대비 바뀐 점, feature/api_db에서 그대로 가져온 것과 바꾼 것(이유 포함)

5 **가공 레이어 (전처리)** 가공
listing_features · recommendable_listings · cutoff_stats · competition_stats 설명과 예시

6 **데이터 갱신·수정 규칙**
언제 자동으로 바뀌는지, 무엇을 고치면 안 되는지, 보정이 필요할 때 하는 방법

7 **직접 해보기**
브랜치 받기 → 전체 수집 → DBeaver에서 보기 → 자주 쓰는 SQL → 컬럼·뷰 추가 방법

8 **수집 범위와 남은 작업**
API에서 다 받았는지, 받지 못한 것과 이유, 앞으로 할 일

1. 한눈에 보기

세 줄 요약

1. jiyoon 브랜치와 feature/api_db 브랜치의 **테이블 구조와 수집 코드를 하나로 병합**했습니다.
2. 청약홈 API를 **2020년~현재 전체 기간**으로 다시 수집하고, 파인 대출 상품과 SH 공고를 추가했습니다.
3. 원본은 그대로 두고, 서비스에서 바로 쓸 수 있게 계산한 **가공 레이어(뷰 4개)**를 만들었습니다. 추천 API가 이걸 씁니다.

데이터 규모

25,939
listings — 공고 × 주택형
(청약홈 공고 6,058건 + SH 99건)

189,785
historical_results — 경쟁률 68,340 · 당첨가점 14,591 · 특별공급 106,854

280 / 682
loan_products / loan_rate_options — 대출 상품 / 금리 옵션

54
recommendable_listings — 지금 접수 예정·접수 중 (공고 13건)

2,344
cutoff_stats — 지역·평형·연도별 커트라인 통계

9,670
competition_stats — 지역·평형·연도별 경쟁률 통계

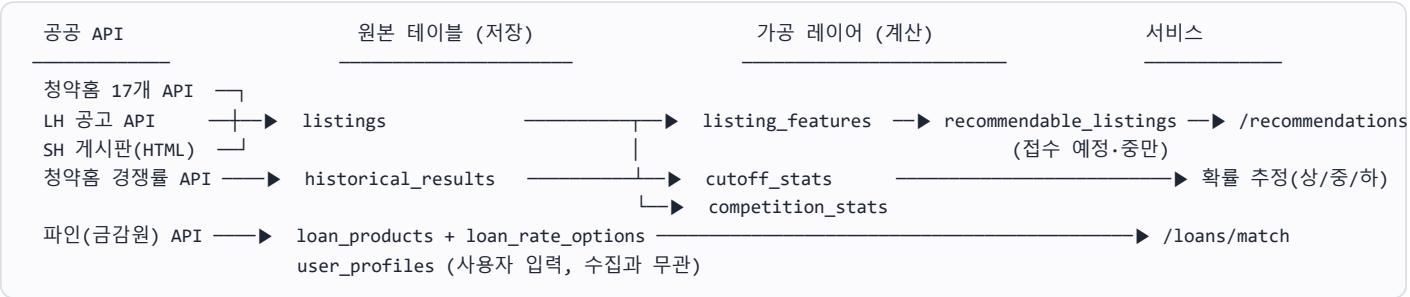
연도별 청약홈 공고 수

연도	2020	2021	2022	2023	2024	2025	2026	합계
공고 수	764	798	1,042	600	920	1,061	873	**6,058**

이번 작업으로 달라진 점

항목	이전	지금
수집 기간	2026년 공고만 (872건)	**2020년 2월 ~ 현재 전체** (6,058건)
접수일·발표일 채움률	45~77% (APT 외 유형 누락)	**100%** (5개 유형 모두)
당첨 커트라인	팀원 표본 0건, 내 DB 936건	**14,591건**
대출 상품	0건	**280개 + 금리 옵션 682개**
수집 시간	공고마다 호출 (수천 번)	1,000행 일괄 조회 **약 150번, 3분**
추천 대상	전체 공고 2.6만 행	**접수 예정·접수 중인 공고만**
합격 확률 추정	동작하지 않음 (컬럼 불일치)	최근 3년 같은 지역·평형 커트라인과 비교해 상/중/하

2. 전체 구조



원본 테이블 = 저장되는 데이터
API가 준 값을 형식만 정리해서 그대로 보관합니다. 수집할 때마다 새 행은 **추가**되고, 기존 행은 바뀐 값만 **갱신**돼요.

가공 레이어 = 계산된 결과
원본을 읽어 파생 변수와 통계를 계산한 **읽기 전용** 뷰입니다. 언제든 다시 만들 수 있고, 결과를 바꾸려면 SQL 정의를 고칩니다.

코드 파일별 역할 (backend/app/)

파일	역할
`data_pipeline/applyhome_client.py`	청약홈 공고 상세 + 주택형 API 호출과 변환 (5개 유형)
`data_pipeline/competition_client.py`	청약홈 경쟁률 · 당첨가점 · 특별공급 신청현황 API 호출과 변환
`data_pipeline/lh_client.py` · `sh_client.py`	LH 공고 API / SH 게시판 HTML 읽기
`data_pipeline/finlife_client.py`	파인 주담대 · 전세 · 신용대출 API 호출과 변환
`data_pipeline/save_listings.py`	**DB 저장은 여기서만.** upsert(있으면 갱신, 없으면 추가), 전체 수집 실행, 수집 후 뷰 갱신
`data_pipeline/scheduler.py`	서버가 켜져 있을 때 자동 수집 (아래 표)
`db/views.py`	**가공 레이어 SQL 정의**, 생성·갱신·재생성 함수
`models/*.py`	원본 테이블 구조 (SQLAlchemy)
`services/probability_estimator.py`	cutoff_stats로 합격 가능성 구간(상/중/하) 추정
`api/routes/recommendations.py`	recommendable_listings의 공고만 추천 대상으로 사용

자동 갱신 주기 (개발 서버가 켜져 있을 때만)

주기	원본에 하는 일	가공 레이어
6시간마다	청약홈 최근 90일 공고 + LH + SH 수집	수집이 끝나면 구체화 뷰 3개를 다시 계산
24시간마다	경쟁률 · 당첨가점 · 특별공급 전체 + 대출 상품 수집	
조회할 때마다	—	`recommendable_listings` 는 저장하지 않고 오늘(한국 날짜) 기준으로 그때 계산

3. 원본 테이블 사전

채움률은 NULL이 아닌 행의 비율입니다. `listings` 는 청약홈 25,840행 기준이에요. **"API에 없음"**은 API가 해당 유형에 그 값을 주지 않는다는 뜻이라 코드로 채울 수 없습니다.

3-1. listings — 청약 공고 × 주택형 원본

행 구분 키: `(source, external_id, model_no)` . 주택형 정보가 없는 공고(LH·SH)는 `model_no` 가 비어 있는 1행으로 저장됩니다.

컬럼	의미	채움	비고
`source`	출처: applyhome / lh / sh	100%	
`external_id` / `model_no`	공고번호 / 공고 안 주택형 순번	100%	LH는 PAN_ID, SH는 게시글 번호
`unit_type`	주택형 (예: 084.7459A = 전용 84.7㎡ A타입)	100%	historical_results와 연결하는 키
`house_type`	APT / 오피스텔·도시형생활주택 / 무순위·잔여세대 / 공공지원민간임대 / 임의공급 / SH·LH 분류	100%	
`complex_name`	단지명 (LH·SH는 공고 제목)	100%	
`region_code` / `region_name`	청약홈 지역코드(100 서울, 410 경기 …) / 지역명	99% / 100%	여러 지역·전국 무순위 공고는 코드 없음
`supply_type`	공급유형 (APT 민영, APT 국민, 신혼희망타운 국민 …)	100%	
`address` / `notice_url` / `homepage_url`	공급 위치 / 청약홈 공고 URL / 분양 홈페이지	100 / 100 / 91%	
`developer` / `constructor` / `contact_phone`	시행사 / 시공사 / 문의처	100 / 55 / 100%	시공사는 APT만 제공
`notice_status`	LH 공고 상태 (공고중 / 접수마감 …)	LH 전용	
`unit_size_sqm` / `exclusive_area_sqm`	공급면적 / 전용면적 (㎡)	63% / 100%	오피스텔·임의공급은 공급면적이 API에 없음
`unit_count`	이 주택형의 공급 세대수 (일반 + 특별)	100%	
`general_supply_count` / `special_supply_count`	일반공급 / 특별공급 세대수	100% / 79%	
`special_supply_detail`	특별공급 종류별 세대수 JSON {"신혼부부": 3, …}	45%	APT·민간임대만 제공
`total_household_count`	공고 전체 공급 세대수	100%	
`sale_price`	분양가 (원). API는 만원 단위	100%	주택형 최고가 기준
`recruit_date`	모집공고일	100%	
`special_receipt_start` / `_end`	특별공급 접수일	49%	특별공급이 있는 공고만
`receipt_start` / `receipt_end`	청약 접수 시작일 / 종료일	100%	추천 대상 판단 기준
`announce_date` / `contract_start` / `contract_end`	당첨자 발표일 / 계약 시작일 / 종료일	100%	
`move_in_date`	입주 예정월 (그 달 1일로 저장)	100%	
`schedule`	순위·지역별 접수일 JSON {"1순위 해당지역": [시작, 종료], …}	57%	APT만 제공
`is_speculative_area` · `is_adjusted_area` · `is_price_capped` · `is_public_housing_district`	투기과열지구 · 조정대상지역 · 분양가상한제 · 공공주택지구 여부	57%	APT만 제공. 자격·대출 규칙에 필요
`eligibility`	자격요건 JSON	빈 값	API에 없는 정보 (8장)

3-2. historical_results — 경쟁률 · 당첨가점 · 특별공급 원본

행 구분 키: `(external_id, unit_type, result_type, category_label)` . `listings` 와는 `external_id + unit_type` 으로 연결됩니다.

컬럼	의미	채움	비고
`result_type`	행 종류: **경쟁률**(5개 유형) / **당첨가점**(APT) / **특별공급**(APT)	100%	68,340 / 14,591 / 106,854행
`external_id` / `unit_type` / `house_type`	공고번호 / 주택형 / 주택 유형	100%	
`category_label`	세부 구분: "해당지역 1순위", "기타지역", "신혼부부 - 기타경기", "일반공급" …	99%	임의공급은 구분 없음
`reside_secd` / `subscrpt_rank`	거주 구분 · 순위(또는 공급종류) 코드 원본	38% / 33%	코드가 있는 유형만

`region_code` / `year`	공고 지역코드 / 당첨자 발표 연도	99.8% / 100%	공고 상세에서 연결해 채움
`exclusive_area_sqm`	주택형에서 추출한 전용면적	96%	
`supply_count` / `apply_count`	공급(배정) 세대수 / 신청 건수	92%	
`competition_rate`	경쟁률. 원본 문자열이 없으면 신청 ÷ 공급으로 계산	82%	미달 행은 비워 둠
`is_undersubscribed`	미달 여부. 특별공급은 세 지역 신청 합계 < 배정일 때	100%	
`raw_cmpet_rate`	API 원본 경쟁률 문자열 ("10.00", "(△209)" = 209세대 미달)	경쟁률 행	검증용
`avg_score` / `cutoff_score` / `top_score`	평균 / 최저(커트라인) / 최고 당첨가점	가점 행	14,591행

3-3. loan_products · loan_rate_options — 대출 상품 원본

loan_products 주요 컬럼	의미
`product_type`	주택담보대출 111 / 전세자금대출 58 / 개인신용대출 111
`fin_group` · `company_name`	은행·저축은행·보험·여신전문 / 회사명
`interest_rate_min` / `_max`	옵션 중 최저·최고 금리 (268개 채움)
`loan_limit_text`	대출 한도 문장 (예: "LTV 70%")
`prepayment_fee_text` · `delay_rate_text` · `extra_cost_text`	중도상환 수수료 · 연체 이자율 · 부대비용 (문장)
`limit_amount`	한도 금액 — 파인이 문장으로만 줘서 비어 있음

loan_rate_options 컬럼	의미
`loan_product_id`	loan_products.id와 연결
`collateral_type`	담보 유형 (아파트 / 아파트 외)
`repay_type`	상환 방식 (분할상환 / 만기일시)
`rate_type`	고정 / 변동 (신용대출: 대출·기준·가산금리)
`rate_min` / `rate_max` / `rate_avg`	최저 / 최고 / 전월 평균 금리(%)
`credit_grade_rates`	신용대출 신용점수 구간별 금리 JSON

`user_profiles`는 사용자가 입력하는 테이블이라 수집과 관계없고, 구조도 바뀌지 않았습니다.

4. 이전 DB와 비교

이전 · jiyoon DB
listings 3,630 / historical 31,394 / 대출 0
2026년 공고 872건, 컬럼 18개

이전 · feature/api_db DB
표본 수집(유형별 5건) 기준
listings 131 / historical 104 / 대출 0, 컬럼 15개

현재 · 병합 DB
listings 25,939 / historical 189,785
대출 280 + 옵션 682, 컬럼 40개

4-1. 테이블 구조를 합친 방식

테이블	기준	합친 내용
`listings`	**feature/api_db의 컬럼 이름** 기준	이름 통일 `hssply_adres` → `address`, `pblanc_url` → `notice_url` 의미 통일 `unit_count` = 일반+특별 (feature/api_db 방식), 일반·특별을 따로 보는 컬럼 추가 키 변경 `external_id` 는 공고번호만. 주택형 순번은 `model_no` 로 분리 (이전: "공고번호-모델번호") 추가 source, 지역명, 시행·시공사, 모집공고일, 특공·계약일, 순위별 일정, 규제지역 여부, 특공 종류별 세대수 등 22개
`historical_results`	**jiyoon 구조** 기준 (경쟁률 상세·가점 3종이 더 풍부)	feature/api_db에서 추가 `region_code`, `year`, 거주·순위 코드 — 확률 추정에 필요 추가 `result_type` (행 종류), `exclusive_area_sqm` 이름 `complex_id` → `external_id`, `house_ty` → `unit_type` (listings와 같은 이름으로 맞춤)
`loan_products`	기존 구조 유지 + 확장	추가 상품 종류, 권역, 회사, 한도·수수료 문장 등 12개 컬럼과 `loan_rate_options` 테이블

4-2. feature/api_db에서 그대로 가져온 것

- **API 호출과 DB 저장의 분리** (`save_listings.py` 에서만 저장) — 전체 구조의 기준으로 삼았습니다.
- 서버 시작·종료 시 스케줄러를 켜고 끄는 처리 (`main.py` 의 lifespan, `stop_scheduler`), 스케줄러 중복 실행 방지
- DB 커넥션 풀 설정 (`session.py`), 자격 필터 임시 로직 (계약금 10% 기준, `eligibility_filter.py`)
- SH 게시판 읽기 코드, 파인·LH 연동 구조, 문서 2개 (`docs/`)

4-3. 바꾼 것과 이유

바꾼 것	이유
청약홈 수집: 공고별 bundle 호출 → 1,000행 일괄 조회	모든 API가 조건 없이 페이지 조회를 지원해서, 전체 기간을 약 150번 호출로 받을 수 있습니다. 하루 호출 한도(API당 10,000건)도 넉넉히 남아요.
커트라인 출처: 당첨자 API → 당첨가점 API	당첨자 API(`ApplyhomeInfoPblancWinnerSvc`)는 "등록되지 않은 서비스"(404)로 응답합니다. 그래서 표본 DB의 커트라인이 0건이었어요.
LH 응답 읽기 (`_extract_rows`)	LH 응답은 `[{"dsSch": …}, {"dsList": […]}]` 형태라, 이전 코드는 이 묶음 두 개를 공고로 읽고 있었습니다.
파인: 은행 주담대 10개 → 3개 상품 × 5개 권역 전체	전세대출은 청약 이후 거주 계획에, 신용대출은 계약금 부족분에 필요할 수 있어서 모두 받았습니다. 금리 옵션도 따로 저장해요.
날짜·필드 처리	유형마다 필드 이름(`RCEPT_*` / `SUBSCRPT_RCEPT_*`)과 날짜 형식(YYYY-MM-DD / YYYYMMDD)이 달라서 둘 다 처리하도록 했습니다.
SH 행	주소를 "서울특별시"로 고정하던 부분은 비워 두고, 게시일은 발표일이 아니라 모집공고일에 넣었습니다.

4-4. 채움률 비교 (주요 컬럼)

컬럼	이전 jiyoon	이전 feature/api_db	현재
listings.receipt_start (접수일)	45%	100%	**100%**
listings.announce_date (발표일)	77%	100%	**100%**
listings.unit_size_sqm (공급면적)	52%	100%*	**63%**
historical.competition_rate (경쟁률)	36%	89%	**82%**
historical.cutoff_score (커트라인)	936행	0행	**14,591행**

* feature/api_db는 공급면적이 없을 때 전용면적 등으로 대체해서 채웠습니다. 현재는 공급면적과 전용면적을 따로 저장해서, 공급면적이 API에 없는 유형(오피스텔·임의공급)은 비어 있어요.

5. 가공 레이어 (전처리)

> 원본을 바꾸지 않고, 서비스가 바로 쓸 수 있게 **계산한 결과**입니다. 정의는 모두 `backend/app/db/views.py` 에 있어요. **구체화 뷰**(materialized view)는 계산 결과를 저장해 두는 뷰로 수집 후에 다시 계산되고, **일반 뷰**는 조회할 때마다 계산됩니다.

5-1. listing_features 구체화 뷰 25,939행

전체 공고(listings)에 **파생 변수**를 붙인 뷰입니다. 추천·비교 화면에서 쓰는 기본 테이블이에요.

추가된 컬럼	계산 방법	쓰는 이유
`area_band`	전용면적 → "60㎡ 이하" / "60~85㎡" / "85㎡ 초과"	청약 가점제·추첨제 비율이 85㎡ 기준으로 달라짐. 같은 평형끼리 비교
`sigungu`	주소의 두 번째 단어 중 시·군·구로 끝나는 것	"해당지역" 자격은 시·군 단위로 판단
`is_private`	supply_type에 "민영"이 있으면 true	가점제는 민영주택, 국민주택은 납입 횟수 기준
`price_per_exclusive_sqm`	분양가 ÷ 전용면적	면적이 다른 주택형끼리 가격 비교
`price_per_pyeong`	분양가 ÷ 공급면적 × 3.3058	업계에서 흔히 쓰는 평당 분양가
`contract_deposit_10pct`	분양가 × 10%	계약금 → 가용자산과 비교, 대출 필요 금액 계산

나머지 컬럼(일정, 세대수, 규제지역 여부, URL 등)은 listings와 같습니다.

5-2. recommendable_listings 일반 뷰 현재 54행 (공고 13건)

접수 예정·접수 중인 공고만 남긴 뷰로, 추천 API는 이 공고들만 대상으로 합니다.

- 조건: `receipt_end ≥ 오늘(한국 날짜)`. 날짜가 지나면 자동으로 빠집니다.
- `receipt_status`: 특별공급(없으면 일반공급) 접수 시작 전이면 "접수 예정", 아니면 "접수 중"
- `days_until_receipt`: 접수 시작까지 남은 날 (음수 = 이미 시작)
- SH는 접수일이 없어서 제외됩니다. LH는 키 문제가 해결되면 마감일 기준으로 포함돼요.

```
충정로역자이르네      084.9972A  60~85㎡  중구    접수 예정  6일 후
강변역 센트럴 아이파크 084.9811C  60~85㎡  광진구  접수 예정  6일 후  평당 38,149,069원
```

5-3. cutoff_stats 구체화 뷰 2,344행

당첨가점 행을 묶은 **커트라인 통계**입니다. 묶는 기준: 지역코드 × 시군구 × 평형 구간 × 민영 여부 × 거주 구분(`reside_label`) × 연도

값: `sample_count`, `notice_count`, `avg_cutoff`, `median_cutoff`, `min_cutoff`, `max_cutoff`, `avg_winner_score`

지역	시군구	평형 구간	거주	연도	표본	평균 커트라인	최저~최고
서울(100)	중랑구	60~85㎡	해당지역	2025	12	57.0	46 ~ 69
서울(100)	서초구	60~85㎡	해당지역	2025	8	71.4	69 ~ 77
서울(100)	동작구	60㎡ 이하	해당지역	2025	7	70.1	66 ~ 74

확률 추정에서 쓰는 방법: 최근 3년 · 같은 지역 · 해당지역 커트라인을 표본 수로 가중 평균합니다. 같은 평형 구간 데이터가 있으면 그걸 먼저 쓰고, 없으면 지역 전체로 넓혀요. 사용자 가점이 평균의 110% 이상이면 "상", 90~110%면 "중", 그 아래면 "하"입니다.

5-4. competition_stats 구체화 뷰 9,670행

경쟁률 행을 묶은 통계입니다. 묶는 기준: 주택 유형 × 지역코드 × 시군구 × 평형 구간 × 세부구분 × 연도

값: `total_supply`, `total_apply`, `overall_rate`(신청 합계 ÷ 공급 합계), `median_rate`, `undersubscribed_ratio`(미달 비율)

유형	지역	시군구	평형 구간	구분	연도	전체 경쟁률	미달 비율
APT	서울	중랑구	60~85㎡	해당지역 1순위	2025	16.10 : 1	0%
APT	서울	중랑구	60㎡ 이하	해당지역 1순위	2025	81.78 : 1	0%
APT	서울	서초구	60~85㎡	해당지역 1순위	2025	170.24 : 1	0%

6. 데이터 갱신·수정 규칙

대상	언제 바뀌나	어떻게 바뀌나
원본 listings, historical_results, 대출	서버가 켜져 있으면 6시간 / 24시간마다, 또는 수동 수집 시	새 행은 추가, 기존 행은 API 값으로 갱신. **예전 행은 지우지 않아서** 데이터가 쌓이기만 합니다.
구체화 뷰 3개	수집이 끝날 때마다	원본을 기준으로 전부 다시 계산
일반 뷰 recommendable_listings	조회할 때마다	그 순간의 한국 날짜로 계산
user_profiles	사용자가 입력할 때	수집과 관계없음

지켜야 할 규칙

> **원본 테이블을 DBeaver에서 직접 고치지 마세요.** API가 주는 컬럼은 다음 수집 때 API 값으로 다시 덮어써집니다.

> **가공 레이어는 읽기 전용입니다.** 결과를 바꾸려면 `db/views.py` 의 SQL을 고친 뒤 `python -m app.db.views` 로 다시 만드세요. 원본은 영향받지 않습니다.

> **사람이 보정해야 하는 값이 있을 때**(예: 자격요건 입력, 잘못된 지역코드 수정)는 보정용 테이블을 따로 만들고, 가공 레이어에서 원본과 합치세요. 그래야 원본과 보정 내용이 모두 남습니다.

DB 데이터는 Git으로 공유되지 않습니다

- Git에 있는 건 **DB를 만드는 코드**(수집 코드, 테이블 구조, 뷰 SQL)입니다. 같은 코드를 실행하면 누구나 같은 DB를 만들 수 있어요.
- 데이터를 바로 나누고 싶으면 `pg_dump` 로 만든 덤프 파일을 공유하거나, 팀 공용 DB 서버를 쓰는 방법이 있습니다.

7. 직접 해보기

7-1. 브랜치 받기 (jiyoon 브랜치가 원격에 push된 뒤)

```
git fetch origin
git switch "jiyoon(cheongyakP)"        # 브랜치 이름에 괄호가 있어서 따옴표 필수
git pull origin "jiyoon(cheongyakP)"
```

7-2. 전체 수집 (약 3분)

`backend/.env`에 `APPLYHOME_API_KEY`와 `FINE_API_KEY`가 있어야 합니다. 테이블 구조가 바뀌었기 때문에, 예전 구조의 테이블이 있다면 먼저 지워야 새 구조로 만들어져요. 이미 있는 테이블은 `create_all`이 바꾸지 않습니다.

```
# DB 컨테이너가 켜진 상태에서 (docker compose up -d)
docker exec cheongyak-platform-db-1 psql -U cheongyak -d cheongyak \
  -c "DROP TABLE IF EXISTS loan_rate_options, loan_products, historical_results, listings CASCADE;"

# backend 컨테이너 안에서 전체 수집 → 테이블·뷰 생성 → 뷰 계산까지 한 번에
docker compose exec backend python -m app.data_pipeline.save_listings
```

7-3. DBeaver에서 보기

- 연결: Host `localhost` · Port `5432` · Database `cheongyak` · User `cheongyak` → 연결 우클릭 → 새로 고침(F5)
- 원본 `cheongyak → Schemas → public → Tables`
- 가공 `… → public → Materialized Views` (listing_features, cutoff_stats, competition_stats) / `… → Views` (recommendable_listings)

7-4. 자주 쓰는 SQL

```
-- 테이블별 행 수
SELECT 'listings', count(*) FROM listings UNION ALL SELECT 'historical_results', count(*) FROM historical_results
UNION ALL SELECT 'loan_products', count(*) FROM loan_products UNION ALL SELECT 'loan_rate_options', count(*) FROM loan_rate_options;

-- 지금 추천 대상 공고
SELECT complex_name, unit_type, area_band, sigungu, sale_price, receipt_status, days_until_receipt
FROM recommendable_listings ORDER BY days_until_receipt;

-- 공고 하나의 주택형별 경쟁률·커트라인
SELECT l.unit_type, h.result_type, h.category_label, h.competition_rate, h.cutoff_score
FROM listings l JOIN historical_results h ON h.external_id = l.external_id AND h.unit_type = l.unit_type
WHERE l.external_id = '2026000202' ORDER BY 1, 2, 3;

-- 서울 60~85㎡ 연도별 해당지역 커트라인 추세
SELECT year, sum(sample_count) AS 표본, round(sum(avg_cutoff * sample_count) / sum(sample_count), 1) AS 평균커트라인
FROM cutoff_stats WHERE region_code = '100' AND area_band = '60~85㎡' AND reside_label = '해당지역'
GROUP BY year ORDER BY year;

-- 금리가 낮은 주담대 옵션
SELECT p.company_name, p.name, o.collateral_type, o.repay_type, o.rate_type, o.rate_min, o.rate_max
FROM loan_products p JOIN loan_rate_options o ON o.loan_product_id = p.id
WHERE p.product_type = '주택담보대출' ORDER BY o.rate_min LIMIT 20;
```

7-5. 컬럼이나 뷰를 추가하고 싶을 때

하고 싶은 것	고칠 곳
API 필드를 원본에 추가	① `models/*.py`에 컬럼 추가 → ② `*_client.py`의 변환 함수(`map_notice`, `map_unit` 등)에 값 추가 → ③ 테이블을 지우고 다시 수집 (Alembic 마이그레이션 도입 전까지)
파생 변수·통계 추가	`db/views.py`의 SQL 수정 → `python -m app.db.views`
수집 주기 변경	`data_pipeline/scheduler.py`

8. 수집 범위와 남은 작업

> **API에서 다 받았나요?** 청약홈과 파인은 제공되는 **데이터 행을 전부** 받았습니다. 다만 일부 필드는 저장하지 않았고, **LH는 키 문제로 0건**, **SH는 공식 API가 없어서 게시판 목록만** 받았어요.

출처	받은 것	받지 않았거나 못 받은 것
청약홈 (API 17개)	전부 공고 상세·주택형·경쟁률 × 5개 유형 + 당첨가점 + 특별공급. 2020년 이후 전체	**일부러 뺀 행:** 점수가 모두 "-"인 당첨가점 15,175행(발표 전·추첨제), 신청·배정이 모두 0인 특공 조합, 지역별 신청 건수가 없는 기관추천 지역 행 **저장하지 않은 필드:** 우편번호, 분양/임대 구분, 정비사업·대규모택지·공공주택특별법 여부, 게재 신문사, 계약면적, 청약신청금, 군(GP), 기관추천 확정·예비 건수, 청약 결과 상태 → 필요하면 7-5 방법으로 추가
파인	전부 주담대·전세·신용 × 5개 권역 × 모든 페이지	여신전문·금융투자 권역은 주담대·전세 상품이 없어 0건. 예적금·연금은 서비스와 관계없어서 제외
LH	0건 수집 코드는 준비 완료	현재 `.env` 키로 LH API 3개가 모두 403(등록되지 않은 서비스키)입니다. LH API를 승인받은 계정의 키로 바꾸고 다시 수집하면 됩니다. 상세·공급정보(주택형, 보증금, 임대료)는 실제 응답을 확인한 뒤 추가해야 해요.
SH	분양·임대 게시판 최근 5페이지 99건	공식 API가 없습니다. 첨부 공고문·본문·오래된 페이지는 저장하지 않았고, 평형·금액은 첨부 PDF에만 있습니다.
API에 없음	—	자격요건(무주택 기간, 소득·자산 기준), 오피스텔·임의공급 공급면적, 추첨제 공고의 가점

남은 작업 (우선순위 순)

#	작업	내용
1	자격요건 규칙	공급유형·규제지역별 1순위 조건(청약통장 가입 기간, 세대주, 재당첨 제한)과 특별공급 소득 기준을 규칙 표로 만들어 `eligibility_filter`에 적용
2	대출 한도 계산	`loan_limit_text`에서 LTV 비율을 추출하고, 분양가와 규제지역 여부로 한도를 계산해 `loan_matcher`에 적용. 정책대출(디딤돌·보금자리론 등)은 직접 입력
3	LH 키 교체	승인 계정 키로 교체 → 공고 목록 수집 → 상세·공급정보 추가
4	API 페이지 나누기	`/listings/` 전체 조회가 27MB라서 pagination이나 검색 조건 필요
5	마이그레이션 도입	테이블 구조가 바뀔 때마다 지우고 다시 수집하지 않도록 Alembic 적용
6	main 반영	팀 검토 후 jiyoon 브랜치 → main PR

수치는 2026-09-30 `cheongyak` DB 조회 결과입니다. 이전 feature/api_db DB 수치는 구조 비교를 위해 `save_all_sources(per_kind=5)`로 만든 표본(9/29)이라, 행 수보다 컬럼 구성을 비교해 주세요.